AWS Zero To Hero - Day 3

IAM Roles, STS and Temporary Credentials - Student Guide | Live Practical

Day 3 Focus
Today we learn how an application running on EC2 securely accesses S3 without storing permanent access keys. We connect trust policies, permission policies, instance profiles, AWS STS and temporary credentials in one practical story.

Code	Meaning
D1-30%	SAA-C03 Domain 1 - Design Secure Architectures (30%)
P1	Operational Excellence
P2	Security
P5	Cost Optimization

Use the green tag near each topic to connect the topic with the exam domain, Well-Architected pillar and AWS security best practice.

Topics Covered

Trust policy vs permission policy - D1-30% | P2 | Best Practice: separate trust from permissions
STS AssumeRole mechanism - D1-30% | P2,P1 | Best Practice: temporary sessions
Instance profiles and EC2 roles - D1-30% | P2 | Best Practice: no keys on servers
Temporary security credentials - D1-30% | P2 | Best Practice: short-lived credentials
Cross-service role pattern - D1-30% | P2,P1 | Best Practice: service roles
Least-privilege S3 practical - D1-30% | P2,P5 | Best Practice: read allowed, write denied

Core Story in Simple Words

Concept	Simple explanation
EC2	A virtual computer running inside AWS.
S3	AWS storage for files and application data.
IAM role	A temporary AWS identity used by a trusted person, service or workload.
Trust policy	Who is allowed to assume this role?
Permission policy	What can the role do after it is assumed?
STS	AWS Security Token Service creates temporary role sessions.
Instance profile	The container used to deliver an IAM role to an EC2 instance.
Temporary credentials	Access key, secret key, session token and expiration time created for a limited session.

Key Line
Trust policy answers WHO. Permission policy answers WHAT. STS supplies temporary credentials. The instance profile delivers the role to EC2.

Trust Policy vs Permission Policy

Policy	Question answered	Day 3 example
Trust policy	Who can assume the role?	EC2 service: ec2.amazonaws.com
Permission policy	What can the role do?	List and read objects from one S3 bucket

Trust policy for EC2

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "ec2.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Permission policy for the demo bucket

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListOnlyTheDemoBucket",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::aws-iam-role-demo-cloudadhar"
    },
    {
      "Sid": "ReadObjectsFromTheDemoBucket",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::aws-iam-role-demo-cloudadhar/*"
    }
  ]
}
```

How AWS evaluates the request

1. The EC2 instance is associated with an instance profile.
2. The instance profile contains EC2-S3-ReadOnly-Role.
3. The role trust policy trusts ec2.amazonaws.com.
4. AWS makes temporary credentials available to the instance.
5. The AWS CLI uses those credentials to call S3.
6. The role permission policy allows or denies the requested S3 action.

Remember

Trust alone does not grant S3 access. Permission alone does not allow EC2 to use the role. Both parts are required.

Day 3 Practical - Exact Resource Names

Resource	Exact name/value
AWS Region	ap-south-1
S3 bucket	aws-iam-role-demo-cloudadhar
Test file	hello.txt
Customer managed policy	EC2-S3-Demo-ReadOnly-Policy
IAM role / instance profile	EC2-S3-ReadOnly-Role
EC2 instance	IAM-Role-Demo-EC2
Security group	IAM-Role-Demo-SG
Network	Default VPC and default public subnet

1. Create the private S3 bucket

```
AWS Console -> S3 -> Buckets -> Create bucket
Bucket name: aws-iam-role-demo-cloudadhar
Region: ap-south-1
Block Public Access: Keep enabled
Default encryption: Keep enabled
```

2. Upload hello.txt

```
Welcome to AWS Zero to Hero.

This file was accessed securely using an EC2 IAM role
and temporary AWS credentials.
```

3. Create the customer managed policy

```
IAM -> Policies -> Create policy -> JSON
Paste the S3 read-only JSON from page 2.
Policy name: EC2-S3-Demo-ReadOnly-Policy
```

4. Create the EC2 role

```
IAM -> Roles -> Create role
Trusted entity type: AWS service
Use case: EC2
Attach: EC2-S3-Demo-ReadOnly-Policy
Role name: EC2-S3-ReadOnly-Role
```

Classroom check

Open the role Permissions tab and Trust relationships tab side by side. Ask: WHO can use the role? WHAT can the role do?

Launch EC2 Without the Role First

Setting	Value
Name	IAM-Role-Demo-EC2
AMI	Amazon Linux 2023
Instance type	A small eligible type such as t3.micro
VPC	Default VPC
Subnet	Any default public subnet
Auto-assign public IP	Enabled
Security group	IAM-Role-Demo-SG
SSH source	My IP
IAM instance profile	None initially

Before attaching the role

```
aws sts get-caller-identity

Expected: Unable to locate credentials.

aws configure list

Expected: access_key and secret_key are <not set>.

aws s3 ls s3://aws-iam-role-demo-cloudadhar/

Expected: Unable to locate credentials.
```

Why this failure matters

The EC2 computer is running, but it has no workload IAM identity. Do not run aws login or aws configure. The goal is to solve the problem with an IAM role.

Attach the role to the running instance

```
EC2 -> Instances -> Select IAM-Role-Demo-EC2
Actions -> Security -> Modify IAM role
Select: EC2-S3-ReadOnly-Role
Choose: Update IAM role
```

Technical note

The console says "attach an IAM role." Technically, EC2 is associated with an instance profile that contains the IAM role.

Verify STS, S3 Access and Least Privilege

1. Verify the assumed-role identity

```
aws sts get-caller-identity

Expected ARN pattern:
arn:aws:sts::<ACCOUNT-ID>:assumed-role/EC2-S3-ReadOnly-Role/<INSTANCE-ID>
```

2. Verify the credential source

```
aws configure list

Expected credential Type: iam-role
```

3. List and download the S3 object

```
aws s3 ls s3://aws-iam-role-demo-cloudadhar/
aws s3 cp s3://aws-iam-role-demo-cloudadhar/hello.txt .
cat hello.txt
```

4. Prove least privilege

```
echo "This upload should fail." > upload-test.txt
aws s3 cp upload-test.txt s3://aws-iam-role-demo-cloudadhar/
```

Expected result

AccessDenied for s3:PutObject. Authentication succeeded because AWS recognized the role. Authorization failed because the permission policy does not allow uploads.

Operation	Required permission	Result
List bucket	s3:ListBucket	Allowed
Download object	s3:GetObject	Allowed
Upload object	s3:PutObject	Denied

Authentication vs authorization

Term	Question	Demo answer
Authentication	Who are you?	EC2-S3-ReadOnly-Role
Authorization	What can you do?	List and download; upload denied

Temporary Credentials with IMDSv2

Get an IMDSv2 token and role name

```
TOKEN=$(curl -sS -X PUT \
  "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

ROLE_NAME=$(curl -sS \
  -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/iam/security-credentials/)

echo "$ROLE_NAME"
```

Show only safe credential metadata

```
curl -sS \
  -H "X-aws-ec2-metadata-token: $TOKEN" \
  "http://169.254.169.254/latest/meta-data/iam/security-credentials/$ROLE_NAME" \
  | jq '{
      Code,
      LastUpdated,
      Type,
      AccessKeyPrefix: (.AccessKeyId[0:4] + "********"),
      Expiration
    }'
```

Security warning

Never display or upload the full AccessKeyId, SecretAccessKey or SessionToken. They are active temporary credentials until they expire.

Cross-Service Role Pattern

```
EC2 service -> EC2 role -> temporary credentials -> S3
Lambda service -> Lambda execution role -> temporary credentials -> S3 or DynamoDB
```

The trusted service changes, but the pattern remains the same: trusted principal -> IAM role -> temporary credentials -> permitted AWS API calls.

Student Assignment

- Create a README.md that explains trust policy vs permission policy.
- Add the permission-policy JSON and trust-policy JSON.
- Capture the failure before attaching the role.
- Capture get-caller-identity after attaching the role.
- Capture S3 read success and S3 upload AccessDenied.
- Draw the EC2 -> Instance Profile -> Role -> STS -> S3 flow.
- Never upload credentials, session tokens, private keys or unmasked account details.

Cleanup

1. Detach the IAM role from EC2.
2. Terminate IAM-Role-Demo-EC2.
3. Delete hello.txt and the S3 bucket.
4. Delete EC2-S3-ReadOnly-Role.
5. Delete EC2-S3-Demo-ReadOnly-Policy.
6. Delete the lab-only security group and key pair if no longer required.

Official AWS References

Topic	URL
IAM roles	https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
IAM roles for EC2	https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/iam-roles-for-amazon-ec2.html
Attach a role to EC2	https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/attach-iam-role.html
Temporary credentials	https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_temp.html